

WaterWise: Co-optimizing Carbon- and Water-Footprint Toward Environmentally Sustainable Cloud Computing

Yankai Jiang
Northeastern University

Rohan Basu Roy
University of Utah

Raghavendra Kanakagiri
IIT Tirupati

Devesh Tiwari
Northeastern University

Abstract

The carbon and water footprint of large-scale computing systems poses serious environmental sustainability risks. In this study, we discover that, unfortunately, carbon and water sustainability are at odds with each other – and, optimizing one alone hurts the other. Toward that goal, we introduce, WaterWise, a novel job scheduler for parallel workloads that intelligently co-optimizes carbon and water footprint to improve the sustainability of geographically distributed data centers.

CCS Concepts: • **Social and professional topics** → **Sustainability**; • **Computer systems organization** → **Cloud computing**.

Keywords: Cloud Computing, Sustainability, Geospatial Shifting

1 Introduction

Problem Motivation and Insights. The rapidly increasing carbon footprint of large-scale computing systems is posing serious environmental sustainability risks – and, has consequently received significant attention from the computer systems research community [2, 5, 15, 24, 25, 27, 31, 51, 54, 58]. Unfortunately, the environmental impact of computing systems on another critical environmental sustainability factor — water footprint – has not been well-explored. A recent study has shown the dramatic side effects that parallel computing workloads, including ML training, place on our water resources [32]. Mitigating the water footprint challenge is critical to making large-scale cloud computing more sustainable. Without a focus on the water footprint of the computing infrastructure, we cannot achieve holistic sustainability of large-scale cloud computing systems.

Motivated by these factors, in this study, we investigate the temporal and spatial interaction between the carbon and water footprints of the computing systems. Our experimental characterization reveals several insights (Sec. 2 and 3). *First, we show that carbon-friendly energy sources (renewable energy sources) reduce the carbon footprint of computing systems – which is desirable, but the availability depends on the geographical location and energy sources may also require water to generate electricity used to power the data centers (offsite water usage).* Unfortunately, some carbon-friendly energy sources (renewable sources) may need proportionally larger amounts of water to generate electricity – *creating a conflict between carbon and water co-optimization* (carbon-friendly energy sources may not be water-sustainable). Furthermore, water is also required to cool down the data centers (onsite water usage) which is significantly impacted by the weather conditions of the data center region – *introducing geographical trade-offs for water sustainability.* Motivated by these insights, we design a carbon- and water-aware job scheduler that aims to navigate these complex trade-offs in a geographically distributed data center serving parallel workloads.

Contributions. The key contributions of WaterWise include the following highlights.

Novel Carbon- and Water-Sustainability Job Scheduling Framework. WaterWise is a novel job scheduler for parallel workloads that identifies the need and opportunity for co-optimizing carbon and water footprint for large-scale systems, and designs a simple yet effective approach to improve the sustainability of geographically distributed data centers. WaterWise reveals that allowing a small amount of delay tolerance can bring additional significant carbon- and water-footprint savings – leveraging the characteristics that batch jobs can inherently tolerate delays [4, 10, 60].

WaterWise Key Ideas and Approach. To navigate the competing carbon and water trade-offs, WaterWise designs and implements a Mixed Integer Linear Programming (MILP) based optimization – due to MILP's inherent known advantage of configurability of objectives, low-overhead, and easier explainability [19]. However, MILP methods as-is cannot operate most effectively in WaterWise's context; to address this limitation, we augment the MILP design with the concept of delay tolerance, soft constraints, and slack management (Sec. 4). WaterWise dynamically and opportunistically exploits spatial and temporal variations in carbon and water intensity characteristics, while ensuring that jobs minimize the violations of their delay tolerance and consider inter-region

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

PPoPP '25, March 1–5, 2025, Las Vegas, NV, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1443-6/25/03.
https://doi.org/10.1145/3710848.3710891

movement transfer latency. Notably, WaterWise recognizes that one unit of water does not have the same value across all geographical regions (one liter of water in a water-stressed region may be more precious than in a water-abundant region [32]). WaterWise systematically captures this effect of water scarcity factor [46] in its scheduling decisions – filling a critical gap in computer system sustainability studies.

Evaluation and Open-source Framework. WaterWise is evaluated using widely-used production traces such as Google Borg [57] and Alibaba [52], and parallel workloads from the PARSEC benchmark [59]. WaterWise provides approx. 21% carbon-footprint and approx. 14% water-footprint savings compared to the carbon- and water-unaware scheduler. WaterWise is available at https://zenodo.org/records/14219862 – the first open-source framework to enable exploration of carbon- and water-aware scheduling.

2 Background

2.1 Carbon Footprint in Data Center

The carbon footprint in the data center consists of two parts: operational and embodied carbon footprint [24].

Operational Carbon Footprint. The operational carbon footprint refers to the carbon emissions incurred while executing the workloads in the data center. The electricity consumed by the data center may come from a mix of energy sources, including both fossil fuels and renewables. The operational carbon footprint can be estimated by multiplying the energy consumption (unit kWh) and the real-time carbon intensity (unit gCO_2/kWh) of the energy mix used by the data center (lower is better). Carbon intensity ($CO_2^{\text{Intensity}}$) depends on the energy mixture of the electricity grid.

Embodied Carbon Footprint. The embodied carbon footprint is a one-time carbon emission that occurs during server design and manufacturing. This footprint is amortized over the hardware's lifespan. Essentially, the footprint attributed to a single job is the server's embodied carbon footprint scaled by the job's execution time over the server's lifetime.

Total Carbon Footprint Estimation. The carbon footprint for a job j is the sum of its operational and embodied carbon footprint and can be expressed as:

$$\begin{aligned} CO_{2,j} &= CO_{2,j}^{\text{operational}} + CO_{2,j}^{\text{embodied}} \\ CO_{2,j} &= E_j \cdot CO_2^{\text{Intensity}} + \frac{t_j}{T_{\text{lifetime}}} \cdot CO_{2,\text{server}}^{\text{embodied}} \end{aligned} \tag{1}$$

Here, $CO_2^{\text{Intensity}}$ represents the real-time carbon intensity of the regional power grid, and it's multiplied by the energy (E_j) of the job to get the operational carbon. t_j denotes the execution time of the job, and T_{lifetime} is the overall lifetime of the server hardware. $CO_{2,\text{server}}^{\text{embodied}}$ is the total embodied carbon footprint of the server. The embodied carbon footprint for job execution is calculated by multiplying the time proportion $\frac{t_j}{T_{\text{lifetime}}}$ by the total embodied carbon footprint of the server.

2.2 Water Footprint in Data Center

In 2023, Google published the data center's water consumption usage of 5.6 billion gallons (equivalent to irrigating 37 golf courses annually) [23], and it observed a 24% increase compared to 2022. This sharp rise highlights the growing reliance on water-intensive cooling technologies to meet computing demands. If this trend continues without innovation in water footprint scheduling techniques, it could exacerbate water stress in regions where data centers are located, particularly in drought-prone areas. Moreover, reducing water usage in energy generation and cooling is crucial for preserving aquatic ecosystems and maintaining biodiversity. Water consumption during energy generation and cooling in the data center can disrupt natural habitats and expose aquatic life to harmful temperature fluctuations and potential pollutants. By minimizing the water footprint, we can lessen these environmental impacts, protect fish populations and other aquatic organisms, and contribute to freshwater ecosystems' overall health and sustainability.

Water Consumption Sources and Water Scarcity. We note that both the indirect usage of water (e.g., water used during electricity generation) and the direct usage of water (e.g., water used during powering and cooling the IT infrastructure) contribute toward the overall water footprint. However, one liter of water is not equal at different geographical locations. This is captured by the metric referred to as **water scarcity**. The *Water Scarcity Factor* (WSF) serves to gauge the degree of water scarcity or stress in specific regions [46], and measure the availability of freshwater relative to demand (a higher WSF indicates that the region is more scarce). To include the impact on region-specific water scarcity, the water footprint is scaled by the WSF factor for the region to reflect the effective water footprint [1, 44]. Next, we provide the basic definitions, intuitions, and estimation methodology for water footprint.

Water Footprint Categories. For simplicity of explanation and estimation, the water footprint can be broken into two parts: **operational water footprint** and **embodied water footprint** [32]. The operational water footprint can be further divided into two categories: *offsite water footprint and onsite water footprint.*

Operational Water Footprint (Offsite). The offsite water footprint is the water usage resulting from using various energy sources to generate electricity for data centers – up to 50% of the water used by data centers can be attributed to offsite usage of water in energy generation [48]. The offsite water footprint depends on the water needed by the energy source to produce electricity, and the total amount of energy generated to be supplied to the data center.

The required amount of water needed to generate a unit amount of energy can vary depending upon the energy mix used to generate the electricity. For example, more offsite

water is needed to generate 1kWh of electricity if the energy source is fossil fuel compared to a solar energy source. This dependence is captured in the *energy water intensity factor* (EWIF) metric. EWIF (unit L/kWh) is dependent on the energy source being used and reflects the intensity of water consumption needed to generate the electricity. Higher EWIF reflects that the energy source is more water-thirsty. If multiple energy sources are used to generate the electricity, then, an average EWIF reflects the water consumption during electricity generation.

The total amount of energy generated to be supplied to the data center depends on the *Power Usage Effectiveness* (PUE) factor. PUE measures the energy efficiency of a data center. Ideally, PUE should be 1.0, but a higher value represents how much energy is spent on non-IT server equipment (e.g., cooling). Therefore, a job with E_j energy uses will need a total of PUE $\times E_j$ energy. The following expression captures the offsite water footprint:

$$\mathrm{H_2O}_j^{\text{offsite}} = \text{PUE} \times E_j \times \text{EWIF} \times (1 + \text{WSF}_r^{\text{dc}}) \tag{2}$$

WSF_r^{dc} is the water scarcity factor in the region where the data center is located and operated. This is included to capture the impact of the water scarcity factor in the region.

Operational Water Footprint (Onsite). Besides water consumption during the electricity generation phase, the data centers directly use water onsite for cooling purposes [14]. The onsite water footprint refers to the amount of water evaporated and dissipated during heat transfer and blowdown in the cooling process. The onsite water footprint can be estimated by multiplying the energy consumed during job execution (E_j) by the *Water Usage Effectiveness* (WUE). The WUE metric quantifies the water required to dissipate heat per unit of energy generated, measured in L/kWh (lower is better). WUE directly depends on the wet bulb temperature specific to the data center's location and is influenced by temporal temperature fluctuations within a region and across regions. This is why two data centers with the same energy-efficiency (PUE) value can have very different onsite water footprints due to differences in the region they are located and the weather in the region over time. The following expression captures the onsite water footprint:

$$\mathrm{H_2O}_j^{\text{onsite}} = E_j \times \text{WUE} \times (1 + \text{WSF}_r^{\text{dc}}) \tag{3}$$

WSF_r^{dc} is the water scarcity factor in the region where the data center is located and operated. This is included to capture the impact of the water scarcity factor in the region.

Embodied Water Footprint. Similar to the embodied carbon footprint, the embodied water footprint is the water consumption generated during server design and manufacturing. Because of the lack of public data on embodied carbon, we use an alternative approach to estimate it. We start by collecting the total embodied carbon footprint of the server [6, 12], and then, multiply it by the carbon intensity of the server's manufacturing place to estimate the total energy consumption during design and manufacturing. Then, this total energy consumption during design and manufacturing can be used to estimate the water consumption during the design and manufacturing – by multiplying the Energy Water Intensity Factor (EWIF) by the total energy consumption during manufacturing ($E_{\text{manufacturing}}$), as expressed below:

$$\mathrm{H_2O}_{\text{server}}^{\text{embodied}} = E_{\text{manufacturing}} \times \text{EWIF} \times (1 + \text{WSF}_r^{\text{server}}) \tag{4}$$

$\text{WSF}_r^{\text{server}}$ is the water scarcity factor in the region where the server is manufactured.

Total Water Footprint. The total water footprint of a job can be expressed as:

$$\begin{aligned} \mathrm{H_2O}_j &= \mathrm{H_2O}_j^{\text{operational}} + \mathrm{H_2O}_j^{\text{embodied}} \\ \mathrm{H_2O}_j &= \text{PUE} \times E_j \times \text{EWIF} \times (1 + \text{WSF}_r^{\text{dc}}) \\ &+ E_j \times \text{WUE} \times (1 + \text{WSF}_r^{\text{dc}}) + \frac{t_j}{T_{\text{lifetime}}} \cdot \mathrm{H_2O}_{\text{server}}^{\text{embodied}} \end{aligned} \tag{5}$$

Water Intensity. *To make the interpretation of results and analysis easier, we propose the concept of water intensity.* Similar to carbon intensity, water intensity captures the spatial and temporal variation in water consumption and the stress it causes on the region (where the data center is located). Intuitively, water intensity metric should be higher (lower is desirable – similar to carbon intensity, where lower is better) if (a) the water scarcity is high (more water stress), (b) water usage effectiveness is high (more water required to dissipate heat per unit of energy generated in the data center, due to unfavorable weather condition), and (c) high PUE and EWIF (i.e., more water is needed - hence, water intensity is higher). Thus, water intensity can be simply captured by focusing on these components in the onsite and offsite water footprint. This can be expressed as:

$$\mathrm{H_2O}^{\text{Intensity}} = (\text{WUE} + \text{PUE} \cdot \text{EWIF}) \cdot (1 + \text{WSF}_r^{\text{dc}}) \tag{6}$$

Note that embodied water footprint is not a part of the water intensity expression because embodied water footprint is relevant only to the region where the server is manufactured, but water intensity is focused on water requirements in the data center region. With this background, next, we discuss the motivations behind WaterWise's design.

3 Experimental Insights and Motivation

Observation 1: *The amount of water required to generate electricity for powering data centers from different energy sources (renewable energy sources vs. fossil energy sources) is significant and varies across energy sources. The water consumption to generate energy from carbon-friendly and carbon-intensive energy sources (resulting in the EWIF factor) varies significantly. In general, carbon-friendly energy sources can have higher water footprint requirements – highlighting the tension between carbon and water as sustainability goals.*

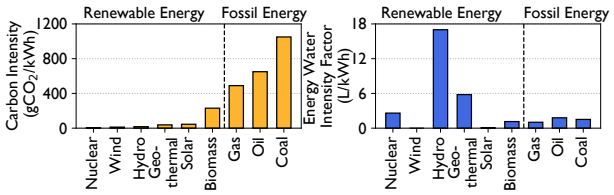


Figure 1. Carbon intensity and water requirements (Energy Water Intensity Factor (EWIF)) for energy generation for different types of energy sources. Carbon-friendly energy sources can have higher water needs for energy generation (EWIF), contributing to a higher offsite water footprint.

Fig. 1 characterizes the notable quantitative differences in carbon intensity among various energy sources and their corresponding EWIF (counts toward offsite water footprint) for power generation. We show that different types of energy sources exhibit varying carbon intensities due to the differing amounts of carbon they produce during power generation [9]. Typically, carbon-intensive energy sources (fossil fuels) have higher carbon intensities compared to carbon-friendly energy sources (renewable energy). For example, coal has a carbon intensity of 1050 gCO_2/kWh is roughly 62× higher than the carbon intensity of hydropower, which is 17 gCO_2/kWh. Interestingly, while renewable energy sources tend to have a lower carbon intensity, they may exhibit higher EWIFs, leading to a greater offsite water footprint. For example, hydropower, despite its low carbon intensity, has a high EWIF of 17L/kWh, which is 11× greater than that of coal. Other renewable energy sources require a significant amount of water for cooling and irrigation, leading to higher EWIF compared to fossil fuels. Other renewable energy sources like biomass require significant water for growing feedstock, which is then converted into energy, and for cooling, leading to higher EWIF compared to fossil fuels. Thus, depending on the energy source mix used by power grids, the carbon and water footprint can vary. Additionally, carbon-efficient energy sources can be water-inefficient, and vice-versa.

Observation 2: *Geographical regions with availability to low carbon intensity energy sources (using more carbon-friendly energy sources for power generation) can be water-stressed regions – due to high EWIF (high water requirements to generate electricity) and high WUE (unfavorable weather conditions requiring more water for cooling). To make it worse, these carbon-friendly but water-stressed regions may have a relatively higher water scarcity factor. Consideration of the water scarcity factor changes the trade-off between carbon and sustainability (carbon vs water intensity).*

From Fig. 2, we observe the average values of different factors impacting the carbon and water footprint of data centers located in different geographical regions. The carbon intensity and the water required to generate electricity to provide energy to a data center (EWIF) varies significantly among different regions due to the difference in the energy sources used by power grids. For example, Zurich (Switzerland) has low carbon intensity (Fig. 2(a)) as the power grid in Zurich uses renewable energy sources including biomass and hydropower [46]. However, since these sources require a high amount of water for generation, the EWIF of Zurich is the highest (Fig. 2(b)), despite having the lowest carbon intensity. Mumbai (India) has a very high carbon intensity as power grids in Mumbai use a large share of non-renewable sources (e.g., coal and oil) to produce electricity. However, since these energy sources have low EWIF (Fig. 1), the regional EWIF is relatively low (Fig. 2(b)).

While EWIF impacts the offsite water footprint, the water usage effectiveness (WUE) impacts the onsite water footprint (Sec. 2.2), required for cooling a data center. Specifically, WUE depends on the wet bulb temperature of a region, which is impacted by the regional temperature and relative humidity. The resulting region-wise WUE variation (Fig. 2(c)) is different from the variations of carbon intensity and EWIF. The difference in region-wise variations of these factors makes it challenging to jointly optimize for carbon and water footprint by distributed job scheduling across multiple regions.

Additionally, from Fig. 2(d), we observe that geographic regions that rely on carbon-friendly energy sources are often regions experiencing high water stress (resulting in high water scarcity factors) – for example, Madrid (Spain). Additionally, certain regions can have low EWIF (power grids use energy sources that consume less water for energy production), but have a high water scarcity (e.g., Mumbai (India) and Oregon (USA)). Thus, in spite of power grids using water-efficient energy sources, it can be unwise to schedule a significant number of jobs in data centers located in water-scarce regions, as it can severely impact the water availability in such regions for other purposes. Combining the water scarcity factor with the water footprint, along with the variations in carbon intensity, makes distributed job scheduling challenging.

From Fig. 2(a)-(d), we observe that the factors impacting the water intensity (EWIF, WUE, and WSF) and carbon intensity vary spatially across regions. However, as the energy mix used by power grids in a region changes with time, they also have temporal variations. This leads to temporal variations in carbon and water intensity, as shown in Fig. 2(e) for Oregon in 2023. Interestingly, we note that certain periods can have high carbon intensity with low water intensity and vice versa. Thus, there is an opportunity to leverage this temporal variation of carbon and water intensity.

Observation 3: *Carbon and water footprint optimization can be performed via effective job scheduling across different data center regions, leveraging temporal variations in carbon and water intensity. The competing nature of carbon and water optimization makes the optimization challenging, but allowing jobs to tolerate some delay increases the opportunity scope.*

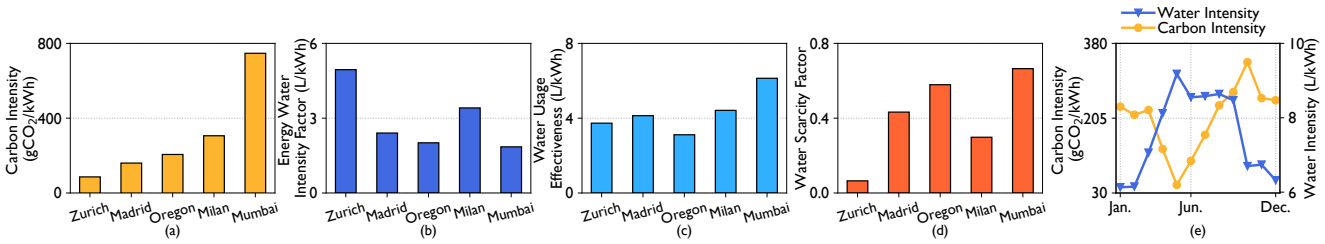


Figure 2. Carbon Intensity, EWIF, WUE, WSF varies across different geographical regions (average values for the year 2023, region labels are sorted according to the carbon intensity). Carbon intensity and water intensity show temporal variations.

To analyze the benefits we observe via distributed scheduling of jobs among different data center regions leveraging temporal variations of carbon intensity and water intensity, we design greedy-optimal solutions: Carbon-Greedy-Optimal, and Water-Greedy-Optimal (referred to as Carbon-Greedy-Opt, and Water-Greedy-Opt and discussed in Sec. 5).

These optimal solutions are infeasible in practice, as they require knowledge about a job's execution time and future variations of carbon and water intensities. We execute jobs following Google Borg trace in five data center regions across the world (details about jobs and execution based on trace in Sec. 5). We observe from Fig. 3(a) that the Carbon-Greedy-Opt solution performs sub-optimally in terms of water footprint and the Water-Greedy-Opt solution is suboptimal in terms of carbon footprint – demonstrating the challenging nature of carbon and water footprint co-optimization. This happens due to the spatio-temporal mismatch of the factors affecting the carbon and the water footprint.

Furthermore, Fig. 3(a) also reveals that if jobs could tolerate some latency in their service time – the opportunity for carbon and water footprint savings improves. This is because delay tolerance allows the scheduler to transfer jobs across regions to opportunistically leverage lower carbon and water intensity. *We define delay tolerance as an allowable % increase in the service time of a job compared to its execution time of the job if it had zero transfer latency and queuing delay.* For example, a 25% delay tolerance indicates the scheduler has the flexibility to delay its execution such that the total service time remains within 1.25× of the execution time if the job was scheduled right away.

To understand the impact of delay tolerance, from Fig. 3(a) we also observe that if delay increases, the carbon and water footprint savings improve even further. When jobs can tolerate higher delay, they can explore more choices of getting transferred to different regions (hiding service time-delay effects). More transfer options to different regions also mean more opportunity to leverage different values of carbon intensity and water intensity, thus resulting in potentially higher carbon and water footprint savings. However, as expected, the returns are diminishing as we increase the tolerance from 10% to 1000%.

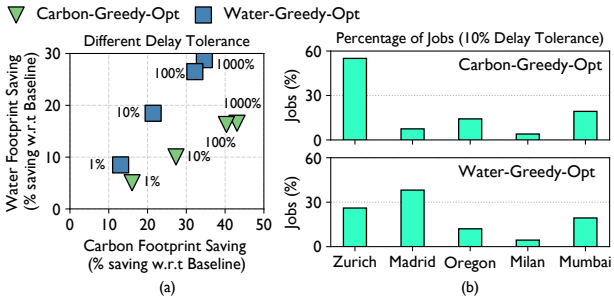


Figure 3. Quantifying optimal solution benefits, opportunity scope due to delay tolerance, and impact on job distribution across regions.

Next, in Fig. 3(b) we observe the percentage distribution of jobs scheduled across the five data center regions following the Carbon-Greedy-Opt and Water-Greedy-Opt strategies, for a 10% delay tolerance. We make two observations. First, as expected, the jobs are distributed across all regions. Even though some regions may be naturally better suited for achieving a carbon- or water-optimal solution, due to temporal variations in carbon and water intensity, jobs are distributed across all regions. This means *no single particular region* is always the optimal choice in terms of minimizing carbon or water footprint. Second, the distributions of jobs across the different regions for carbon and water-optimal solutions are significantly different.

4 WaterWise Design

WaterWise Objective and Desirable Design Goals. The primary objective of WaterWise is to minimize the carbon footprint and water footprint when executing jobs in geographically distributed data centers, under different specified levels of delay tolerance. The system design of WaterWise should strive to meet several desirable requirements. In particular, the core scheduler engine of WaterWise should be configurable to balance different objectives, the decisions should be interpretable, the decision-making overhead should be low, and the delay tolerance limit should not be violated for jobs. While the scheduler cannot have futuristic information (e.g., future carbon and water intensity for different regions), it should co-optimize jobs that are invoked together or nearby in time based on the current carbon and water intensity of all available regions.

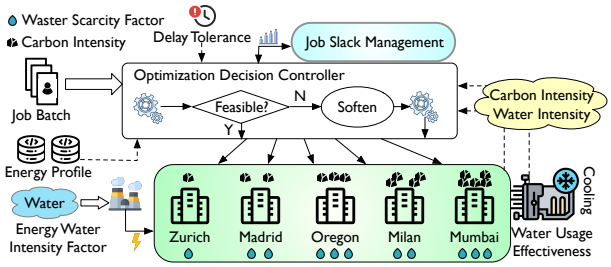


Figure 4. Design overview of WaterWise.

System Overview of WaterWise. WaterWise collects and holds all detailed information about the incoming jobs, including job dependencies, execution files, and metadata. Each job specifies a home region, indicating where the users have submitted their jobs. If WaterWise determines to move the job away from its home region, it considers the transfer latency impact on the service time and allowable delay tolerance. WaterWise Optimization Decision Controller decides where to execute which job while considering the current carbon and water intensity of all regions and collected current mean estimates about job execution time and energy from their previous executions; however, these estimates can be inaccurate.

Recall that, as desirable properties, the core scheduler engine of WaterWise should be configurable to balance different objectives, the decisions should be interpretable, and the decision-making overhead should be low. Therefore, WaterWise employs a Mixed Integer Linear Programming (MILP) based approach for scheduler optimization. While other approaches are certainly viable (e.g., machine learning) and potentially effective, WaterWise chose MILP to keep the overhead practical (as confirmed in evaluation) and decisions interpretable/scheduling objectives configurable, as discussed shortly. WaterWise's Optimization Decision Controller dispatches jobs to selected regions and reconfigures the job queue to ensure that jobs with the least slack in meeting their delay tolerance are prioritized. The overview of WaterWise is shown in Fig. 4.

WaterWise Objective Function. Recall that carbon and water are often conflicting goals both in time and space, and WaterWise's goal is to co-optimize both the carbon and water footprint considering all the regions and all available jobs ready to be executed. Formally, the objective function can be expressed as:

$$f(j,r) = \lambda_{CO_2} \frac{CO_2(j,r)}{CO_{2,j}^{max}} + \lambda_{H_2O} \frac{H_2O(j,r)}{H_2O_j^{max}} \tag{7}$$

Here, $f(j,r)$ is the optimization objective function, j is the job to be scheduled and r is the region for execution selected from the available regions. λ_{CO_2} and λ_{H_2O} are configurable parameters that the cloud service provider can set to control the optimization degree of carbon and water footprint, respectively. The sum of these parameters equals one. $CO_2(j,r)$ and $H_2O(j,r)$ represent the carbon footprint and water footprint for executing the job j in the region r. $CO_{2,j}^{max}$ and $H_2O_j^{max}$ are the maximum carbon and water footprint of job j executed in the region with the highest carbon intensity and water intensity, respectively. WaterWise leverages these maximum values for normalization to ensure that one objective does not skew the optimization. For simplicity, we assume that data centers in different regions use the same generation of hardware with no performance differences, while transfer latency between regions is considered in the optimization process as described below.

Problem Formulation. We now formulate this job scheduling problem using Mixed Integer Linear Programming (MILP). The inputs of WaterWise are M jobs ready for execution, and data centers are distributed across N different regions. WaterWise uses one optimization variable, denoted as $x_{m,n}$. Here, m and n indicate the m-th job and n-th region, and $x_{m,n}$ is a Boolean variable, where $x_{m,n} = 1$ indicates that the m-th job is assigned to the data center in the region n for execution, and $x_{m,n} = 0$ indicates otherwise. Our goal is to minimize the objective function defined in Eq. 7, as $f(m,n)$. This problem can be formulated as:

$$\min \sum_{m=1}^{M} \sum_{n=1}^{N} x_{m,n} \cdot [\lambda_{CO_2} \frac{CO_2(m,n)}{CO_{2,j}^{max}} + \lambda_{H_2O} \frac{H_2O(m,n)}{H_2O_j^{max}} + \lambda_{ref} \cdot (\lambda_{CO_2} \cdot CO_{2,n}^{ref} + \lambda_{H_2O} \cdot H_2O_n^{ref})] \tag{8}$$

WaterWise leverages the historical carbon footprint and water footprint (normalized) of every region in a time window as the history learner to the objective function ($CO_{2,n}^{ref}$ and $H_2O_n^{ref}$). WaterWise utilizes configurable parameters (λ_{ref}) to control the influence of history on the optimization process.

The optimization variable ($x_{m,n}$) must satisfy two constraints. First, WaterWise ensures that each job is assigned to only one data center in a specific region for execution only if the capacity is available (Eq. 9). Second, each region has a limited capacity. The number of jobs assigned to any region cannot exceed its capacity (Eq. 10). We use $cap(n)$ denoting the remaining capacity of the data centers in region n. The remaining capacity is gathered using the monitor in WaterWise. We formalize these 2 constraints as the following:

$$s.t. \quad \sum_{n=1}^{N} x_{m,n} = 1, \quad \forall m \in \{1,2,\ldots,M\} \tag{9}$$

$$s.t. \quad \sum_{m=1}^{M} x_{m,n} \leq cap(n), \quad \forall n \in \{1,2,\ldots,N\} \tag{10}$$

These two constraints must be satisfied. If the capacity is not enough among all regions combined, WaterWise uses its slack management to prioritize certain jobs to avoid violating delay tolerance.

Delay Tolerance Constraint. Recall that delay tolerance, a configurable parameter for the service provider, determines the maximum allowable service time for a job (relative to its

pure execution time in its home region - that is, no queuing or transfer latency). The execution time of a single job is gathered from the database, and the maximum service time allowance is calculated based on this execution time. Delay tolerance (denoted as TOL%) specifies the percentage by which the total service time (including transfer latency and queuing delay) can exceed the execution time. The delay constraint can be expressed as:

$$s.t. \quad \sum_{n=1}^{N} x_{m,n} \cdot \frac{L_{m,n}}{t_{m,n}} \leq (\text{TOL}\%), \quad \forall m \in \{1, 2, \ldots, M\} \tag{11}$$

$L_{m,n}$ is the transfer latency when transferring m-th job from its original region to n-th region. $t_{m,n}$ denotes the execution time of m-th job in n-th region. Note that, different regions utilize the same type of hardware, so there are not many performance variations. Higher delay tolerance allows WaterWise to achieve greater carbon and water savings.

MILP Optimization. The problem formulation is Eq. 8 with the constraints (Eq. 9, 10, 11). WaterWise solves the MILP problem with PuLP [41] to determine the global optimal scheduling decisions of all jobs. PuLP is a Python package integrating multiple algorithms (cutting plane, branch-and-bound) for solving the MILP problem. Its optimization results are equally accurate and the optimization process is more configurable compared with the heuristic algorithms (genetic algorithm, anneal simulated algorithm), also unlike machine learning models, where the reasoning behind outcomes might be opaque, MILP outcomes can be traced directly back to the mathematical model. Details of the solver and decision-making overhead are in Sec. 5 and Sec. 6. However, MILP solver has two challenges in WaterWise's context: (a). *Fragility under hard constraints*: MILP solver can fail to provide a solution if all the constraints cannot be satisfied, making it less flexible in certain situations. (b). *Statelessness w.r.t. delay tolerance violation*: MILP problem itself is stateless, it does not retain information about which jobs may be closer to delay tolerance. The solver only solves the problem given the required parameters, it doesn't have the awareness of the potential for delay tolerance violation if the job arrived in the past and has been delayed in anticipation of better carbon and water savings. To address these issues, we show two extensions next in the Decision Controller to make WaterWise effective and robust and meet the design properties.

WaterWise's Soft Constraints. When the solver cannot provide feasible and robust optimization outputs, WaterWise softens the delay tolerance constraint (Eq. 11) based on the penalty methods [8]. The penalty method is an optimization technique used to solve constrained problems by converting them into unconstrained ones. A penalty term is added to the objective problem, which imposes a cost for violating the constraints. The magnitude of the penalty increases as the solution moves further from satisfying the constraints, thereby guiding the optimization process toward near-optimal solutions. By adjusting the penalty value, the delay penalty can balance between optimizing the objective and adhering to the constraints, making WaterWise effective.

Algorithm 1 WaterWise's Scheduling Framework

Input: Job Batch: $J = \{j_m | m \in [1, M]\}$.
Output: Decisions D.
Get the all jobs that need to be scheduled $J_{all} = J \cup J_{\text{delay}}$.
Get energy (E_j), execution time (t_j), latency, capacity ($\text{cap}(n)$).
if M> $\sum_n \text{cap}(n)$ **then**
    Use WaterWise slack manager to pick top $\sum_n \text{cap}(n)$ jobs.
    Use the WaterWise soft constraint enabled Decision Controller: D.
else
    Use the WaterWise hard constraint enabled Decision Controller: D.
    if D is not a feasible solution **then**
        Use the WaterWise soft constraint enabled Decision Controller: D.

For more details, WaterWise converts the delay tolerance constraint (Eq. 11) into a new relaxed constraint (Eq. 13). This relaxation is achieved by introducing relaxed penalty variables that account for potential violations of the original delay tolerance. The penalty associated with these variables is then incorporated into the objective problem. The modified optimization problem can now be expressed as follows:

$$\begin{aligned} \min \sum_{m=1}^{M} \sum_{n=1}^{N} x_{m,n} \cdot [\lambda_{CO_2} \frac{CO_2(m,n)}{CO_{2,j}^{\max}} + \lambda_{H_2O} \frac{H_2O(m,n)}{H_2O_j^{\max}} \\ + \lambda_{ref} \cdot (\lambda_{CO_2} \cdot CO_{2,n}^{ref} + \lambda_{H_2O} \cdot H_2O_n^{ref})] + \sigma \cdot \sum_{m=1}^{M} \sum_{n=1}^{N} P_{m,n} \end{aligned} \tag{12}$$

$$s.t. \quad \sum_{n=1}^{N} x_{m,n} \cdot \frac{L_{m,n}}{t_{m,n}} \leq (\text{TOL}\% + P_{m,n}), \quad \forall m \in \{1, 2, \ldots, M\} \tag{13}$$

Job Slack Management. WaterWise designs slack management to determine which jobs are closer to their respective delay tolerance violation (recall that MILP does not retain state information across interactions). WaterWise's slack manager determines which jobs should be prioritized for scheduling and where they should be executed. Once these prioritized jobs are scheduled, any remaining unscheduled jobs will be considered in the next iteration.

More formally, when the number of jobs exceeds the total available capacity $\sum_{n=1}^{N} \text{cap}(n)$, meaning the current cluster cannot handle this many jobs, WaterWise composes the following urgency score to determine the priority of each job:

$$\text{Urgency} = \text{TOL}\% \cdot t_m - L_m^{\text{avg}} - (T_m^{\text{start}} - T^{\text{current}}) \tag{14}$$

WaterWise ranks all the jobs based on the urgency score from smaller to higher. To meet the delay tolerance constraint, WaterWise prioritizes the jobs according to the urgency score. $\text{TOL}\% \cdot t_m$ is the maximum allowable service time

Table 1. Benchmark workloads used in WaterWise.

Name	Benchmarks
PARSEC	Dedup, Netdedup (Data Compression); Canneal (Engineering); Blackscholes, Swaptions (Financial Analysis)
CloudSuite	Data Caching; Graph Analytics; Web Serving; Memory Analytics; Media Streaming

for serving m-th job. L_m^{avg} is transfer latency across all available regions. T_m^{start} is the time when the controller receives the m-th job, and $T^{current}$ is the current time when calculating the urgency score. The last component $(T_m^{start} - T^{current})$ illustrates how long the job has been waiting.

Summary. The scheduling framework is summarized in Algorithm 1. Next, we provide the evaluation of WaterWise's effectiveness and robustness.

5 Experiment Methodology

Experimental Setup. All experimental measurements are performed on a 175-node cluster containing machines equally distributed across five different AWS regions: `eu-central-2` (Zurich), `us-west-2` (Oregon), `eu-south-2` (Spain), `eu-south-1` (Milan), `ap-south-1` (Mumbai). The servers are AWS `m5.metal` general purpose bare-metal machines, each equipped with four 24-core, 384 GiB memory Intel Xeon Platinum 8175 CPUs with 25 Gibps network. The execution files and dependencies of jobs are compressed into `.tar`, and transferred across regions using `SCP`.

The WaterWise's Decision Controller is driven by a module leveraging PuLP [41] - a Python library for MILP, offering a high-level configurable interface to define optimization problems. The optimization objective is solved using various GLPK solver, which adopts the cutting plane and branch-and-bound algorithms. In our default setting, WaterWise assigns equal weight (0.5) to both λ_{CO_2} and λ_{H_2O} to optimize carbon and water footprint, and a weight of 0.1 to λ_{ref} in the History Learner with a window size of 10; these parameters are configurable and were not hand-tuned to achieve better performance. The delay tolerance ranges from 25% to 100%, and we set the PUE to 1.2 [47]. Our full experimental framework including the experimental measurements, experimental methodology, codebase for trace-simulation, and experiments will be open-sourced.

Evaluated Workloads and Traces. WaterWise is evaluated using a wide range of ten benchmarks from the CloudSuite benchmark suite [18] and the PARSEC-3.0 benchmark suite [59]. The selected benchmarks represent a wide range of scientific domains, such as graph analysis, machine learning, parallel computing, etc (Table 1). When executing jobs, we ensure that job inter-arrival follows the industry-grade traces from Google (Borg cluster traces [57]) to capture the characteristics observed on production-scale cloud computing platforms. Our evaluated trace contains ten-day data from the Google trace, including more than 230000 jobs. In the evaluation (Sec. 6), we evaluate WaterWise with another widely-used cloud trace, Alibaba VM Cloud trace [52] to further demonstrate the effectiveness of WaterWise. The average utilization of the evaluated cluster setup is approx. 15% (based on the Borg cluster traces and number of servers used in the setup) – aligned well with widely-reported data center utilization trends [11, 30], but we also perform sensitivity study to demonstrate WaterWise's at both higher (25%) and lower (5%) utilization levels in Sec. 6.

Carbon, Water Footprint Estimation. Carbon and water footprint estimation follows the formulas outlined in Sec. 2. Energy consumption is measured using the Likwid [53] to read the RAPL [29] energy information. WaterWise utilize a publicly available dataset [13] to determine the total embodied carbon footprint of the `m5.metal` instance. Additionally, WaterWise uses carbon intensity data from July 2023 across five different regions, sourced from Electricity Maps [38].

The Energy Water Intensity Factor (EWIF) of each energy source is obtained from a widely-used open-source dataset source [35, 36]. The EWIF of energy sources influences the water intensity estimation; therefore, we also conduct a robustness analysis in Sec. 6. Electricity Maps is used to feed the real-time breakdown information of energy mix for each region. We also evaluate the effectiveness of WaterWise when the recent data [45] from World Resources Institute is used. Based on the energy mix breakdown and the EWIF of individual energy sources, WaterWise calculates the regional EWIF for estimating the offsite water footprint. For onsite water footprint, WaterWise adopts the web bulb temperate of a data center's region from Meteologix [39] to determine the Water Usage Effectiveness (WUE) [32]. The Water Stress Factor (WSF) of different data center regions is collected from an open-source dataset [1].

Relevant Techniques. We compare WaterWise with the following scheduling schemes:

Baseline. The baseline scheme contains a vanilla scheduling framework, every job is executed in its home region. This scheme does not explore the potential of carbon and water savings via migration to other locations or opportunistically delaying the execution at home locations in anticipation of carbon or water savings.

Carbon-Greedy-Opt, Water-Greedy-Opt. Both schemes are two infeasible schemes that only optimize either carbon or water footprint, respectively. Both schemes have the future knowledge of carbon intensity and water intensity for each region, jobs can be scheduled later to get more carbon and water savings. Note that, the maximum delay is bound by the delay tolerance constraint. Both schemes can have varied carbon and water footprints due to different delay tolerances. We use the brute force method to find the overall carbon and water footprint. These methods are not truly optimal since they make the scheduling decision without knowing the characteristics of future job arrivals.

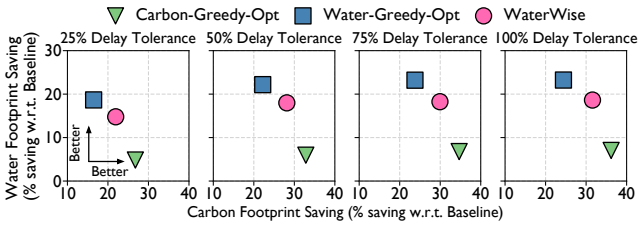


Figure 5. WaterWise provides significant carbon and water footprint savings compared to the baseline. Higher delay tolerance can further improve carbon and water footprint saving.

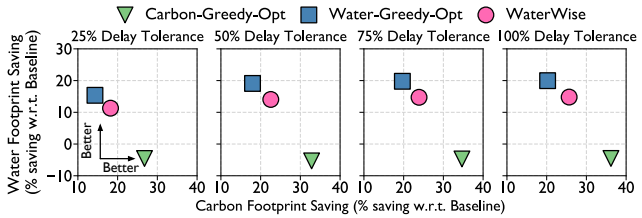


Figure 6. WaterWise consistently provides significant carbon and water footprint savings compared to the baseline, when evaluated with recent World Resources Institute data [45].

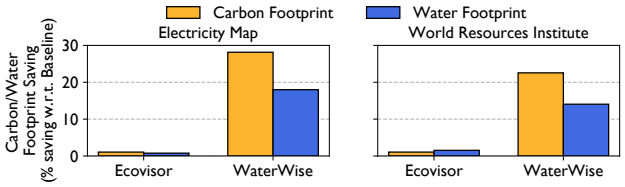


Figure 7. WaterWise yields higher carbon and water footprint savings compared to Ecovisor when evaluated with Electricity Map and World Resources Institute data.

Round-Robin and Least-Load Techniques. Round-Robin and Least-Load are two common load-balancing algorithms used to distribute incoming parallel jobs across multiple servers to optimize resource utilization and ensure high availability. The Round-Robin works by sequentially distributing jobs to each server across regions, one after the other, in a circular order. The Least-Load algorithm dynamically distributes parallel jobs based on the current load of each region. It sends the job to the region with the lowest resource usage, ensuring a more balanced distribution. Both of these algorithms are not aware of the carbon and water savings.

Ecovisor. Ecovisor is a state-of-the-art virtualization container designed to execute applications with carbon efficiency [50]. Ecovisor is a novel technique that uses the carbon scaler to control the power efficiency of the jobs. Ecovisor operates by leveraging solar energy, grid power, and a virtual battery to manage the power usage of the container. We note that Ecoviser only targets the carbon footprint metric (but not the water footprint), so the scope of WaterWise and Ecovisor are fundamentally different. Nevertheless, we include Ecovisor to provide a more comprehensive evaluation to the readers. We use a customized implementation of Ecovisor [50] to enhance its functionality and use the setting of WaterWise. Refer to Sec. 6 for more details about the comparison with WaterWise.

Figures of Merit. The primary metrics in WaterWise are the carbon footprint and water footprint as defined in Sec. 2. They are represented as percentages of savings compared to the baseline (higher is better). We also report average execution time and the number of jobs with delay tolerance violations (as % of all jobs), to better understand the performance, carbon, and water trade-offs, and WaterWise's effectiveness.

6 Evaluation

WaterWise provides significant carbon and water footprint savings, while operating under a delay tolerance constraint. Fig. 5 shows the effectiveness of WaterWise compared to Carbon-Greedy-Opt and Water-Greedy-Opt for different delay tolerances of service time when executing jobs following the Google Borg trace.

We observe that WaterWise consistently balances between carbon footprint optimization and water footprint optimization and results in only 6.62% carbon footprint increase with respect to Carbon-Greedy-Opt and 4.77% water footprint increase with respect to Water-Greedy-Opt. As we increase the delay tolerance, jobs get more opportunities to be scheduled in remote data center locations for their execution, which increases their chances of further reducing carbon and water footprint. Thus, with an increase in delay tolerance, both WaterWise's and Carbon-Greedy-Opt/Water-Greedy-Opt's performance improves. As the delay tolerance increases from 25% to 100%, the carbon footprint of WaterWise improves by 12.37% and the water footprint improves by 4.52%. Note that, WaterWise always performs better than the baseline solution by at least 21.91% and 14.78% in terms of carbon and water footprint, respectively, as the baseline solution always schedules jobs in their home data center regions in carbon-unaware and water-unaware fashion.

We acknowledge that embodied carbon footprint and water intensity calculation can have small inaccuracies or variations because the estimation relies on the accuracy of external data sources and methodology is not widely standardized yet (e.g., not all manufacturers report the embodied carbon consistently and not in the same way). Nevertheless, our sensitivity analysis shows that even with a 10% variation in the estimation of embodied carbon and water intensity, WaterWise can still provide benefits. Specifically, WaterWise achieves 18% and 26% reduction in carbon and water footprint compared to the baseline solution, under

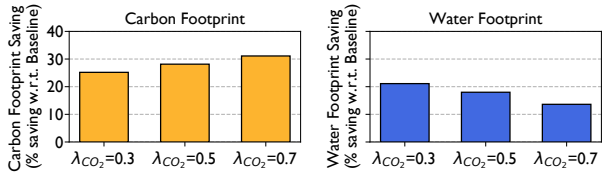


Figure 8. WaterWise is configurable and effective as the optimization weight factor for carbon and water footprint changes.

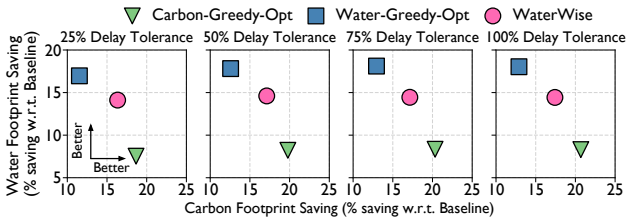


Figure 9. WaterWise's effectiveness with the Alibaba trace.

a 10% variation in embodied carbon and 50% delay tolerance. Additionally, WaterWise reduces 28% carbon footprint and 18% water footprint within 50% delay tolerance when subjected to a 10% variation in water intensity.

In Fig. 5, we used the energy mix breakdown data from the Electricity Maps [38] in all the evaluations. To evaluate the sensitivity of the water intensity data, we also compare using a different external dataset [45] from the World Resources Institute for calculating the water intensity in Fig. 6. WaterWise yields over 18% carbon footprint and 11% water footprint savings compared to the baseline.

In Fig. 7, we compare WaterWise with Ecovisor [50]. As discussed in Sec.5, Ecovisor focuses only on optimizing the carbon footprint, and not the water footprint — a key focus of WaterWise. The carbon scaler in Ecovisor adopts a flexible strategy that dynamically adjusts container resources, scaling up or down to maintain a consistent carbon footprint for long-running jobs, when the carbon intensity fluctuates. However, if the initial carbon intensity is high when the experiment begins, the target carbon footprint is always set high, resulting in an overall elevated carbon footprint for the job. Unlike WaterWise, Ecovisor primarily focuses on operational carbon during optimization, and the embodied carbon is not as thoroughly considered as WaterWise. Notably, embodied carbon is closely tied to the execution time of jobs, making it an essential component of comprehensive carbon-aware scheduling. Moreover, in the setting of Ecovisor, every job is executed in their home regions using Ecovisor, and Ecovisor does not explore cross-regional scheduling opportunities to reduce both carbon and water footprints at the same. Therefore, while Ecovisor is novel and effective for the purpose it was designed, it achieves modest carbon and water savings compared to the baseline scheme (but less than WaterWise). For example, using Electricity Map data, WaterWise reduces the carbon footprint by 27.6% and the water footprint by 17.46% compared to Ecovisor.

Table 2. Average service time and delay tolerance violations.

	Service Time (norm. to execution time)				% Job Violation			
Delay Tolerance	25%	50%	75%	100%	25%	50%	75%	100%
Baseline	1×	1×	1×	1×	0%	0%	0%	0%
Carbon-Greedy-Opt	1.06×	1.18×	1.28×	1.50×	1.1%	0.2%	0.3%	0.5%
Water-Greedy-Opt	1.07×	1.17×	1.23×	1.34×	1.7%	0.1%	0.2%	0.2%
WaterWise	1.03×	1.09×	1.11×	1.13×	2.5%	0.03%	0.03%	0.02%

WaterWise remains effective even when the optimization weights for carbon and water footprints are reconfigured to be unequal. In Fig. 8, when carbon footprint minimization is given 30% (λ_{CO_2} = 0.3). WaterWise can still reduce 25.18% carbon and 21.1% water, within 50% delay tolerance. Conversely, if carbon footprint minimization is given 70% (λ_{CO_2} = 0.7). WaterWise can still reduce 31.1% carbon and 13.6% water compared to the baseline scheme, again within 50% delay tolerance.

WaterWise is designed to adapt to different request patterns and rates of uploading jobs. For example, even if the request rates in the Google Borg trace double compared to normal rate levels, WaterWise reduces carbon footprint by 21.7% and water usage by 10.2% compared to the baseline scheme. Additionally, Fig. 9 shows similar trends in WaterWise's performance when the jobs are invoked following the Alibaba Cloud VM trace with different request patterns. For example, on average, for a 25% delay tolerance WaterWise performs within 3.43% and 2.85% of Carbon-Greedy-Opt and Water-Greedy-Opt in terms of carbon and water footprint, respectively.

Recall that the major components of WaterWise's design are the MILP solver and the slack manager, each of which plays a key role in WaterWise's effectiveness. Based on the constraints of data center capacity in different regions and delay tolerances, WaterWise's MILP solver schedules jobs to minimize carbon and water footprint, which is why WaterWise's performance remains close to the optimal solutions. The slack manager considers the queue wait delay and transfer latency of jobs to different data center regions and accordingly forms solution constraints for the MILP solver. This is why, very few jobs violate the delay tolerance constraints as seen in Table 2. In rare cases, where no feasible solution can be obtained by the MILP solver, the constraints are softened, which results in a few delay tolerance violations. As expected, the violations decrease with an increase in delay tolerance. Note that, these violations are not an artifact of WaterWise's design – in fact, even the optimal solutions experience these delay violations (Table 2). They occur due to the limited capacity of data centers.

Additionally, from Table 2 we also observe that the average service time is much lower than the delay tolerance. For example, for a 25%, 50%, 75%, and 100% service time delay tolerance compared to the execution time, the average delay of jobs is only 3%, 9%, 11%, and 13%, respectively. This delay arises from the transfer latency of jobs to remote data centers and the queuing delay when data centers have no

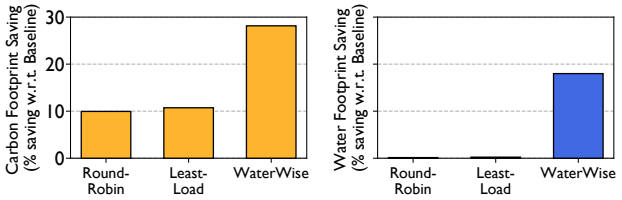


Figure 10. WaterWise's comparison with other alternatives.

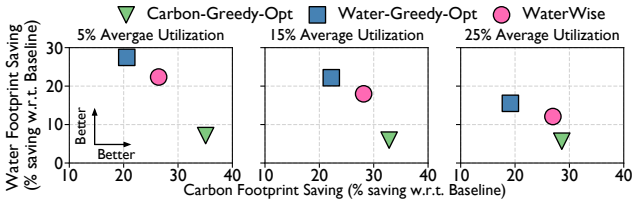


Figure 11. WaterWise across different utilization levels.

capacity for job execution. WaterWise's slack manager designs constraints so that the MILP solver schedules jobs to minimize carbon and water footprint, without much increase in service time.

Constraint-guided optimization of WaterWise to schedule jobs, helps it to obtain better solutions than other possible choices of scheduler design. In Fig. 10, we compare the effectiveness of WaterWise with a scheduler that picks the data center region for job execution in a Round-Robin manner (Round-Robin), and one which schedules jobs in data center region with the least load (Least-Load). WaterWise performs scheduling based on the temporal and spatial variations of the factors affecting the carbon and water footprint (carbon intensity, EWIF, WUE, and WSF). This helps it to optimize carbon and water footprint by at least more than 19.5%, and 17.8%, respectively, compared to these carbon and water footprint-unaware scheduling solutions.

All the previous results showed WaterWise's performance with an average of 15% server utilization in the different data center regions. In Fig. 11, we show that WaterWise remains effective across different average utilization levels, which are obtained by changing the number of available servers in various data center regions. For example, with a 5% utilization, WaterWise performs with 13.31% and 7.04% of Carbon-Greedy-Opt and Water-Greedy-Opt in terms of carbon and water footprint, respectively. The capacity constraint changes based on the number of available servers with different data center utilization percentages. Based on this constraint, WaterWise's MILP solver finds the optimal server for job execution.

WaterWise can successfully adapt to changes in resource availability and remain effective. Fig. 12 shows the results when we remove regions from the original evaluation setting. Different region availabilities can influence the carbon and water savings, due to the variance of carbon and water intensities. For example, the availability of Mumbai enables significant carbon savings, this is because WaterWise

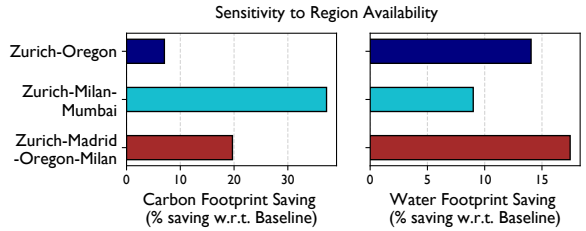


Figure 12. WaterWise is effective in saving carbon and water footprints under different resource availability.

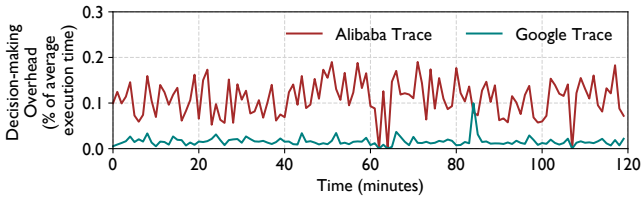


Figure 13. WaterWise demonstrates low overhead.

Table 3. Communication overhead (the home region is Oregon).

Regions	Avg Carbon Overhead (% Execution Carbon)	Avg Water Overhead (% Execution Water)
Zurich	0.08	0.09
Madrid	0.08	0.09
Milan	0.13	0.09
Mumbai	0.17	0.13

can explore the variances of carbon intensities across these 3 regions (Zurich, Milan, Mumbai). A job submitted from Mumbai (high carbon intensity region) can save carbon footprint, if this job is scheduled to Zurich (low carbon intensity region).

Finally, we demonstrate that the decision-making overhead of WaterWise is practical and minimal. Fig. 13 shows that WaterWise decision-making overhead is negligible – less than 0.2% of the average execution time of the jobs. This is because WaterWise uses MILP solver which minimizes overhead by the use of solutions such as branch-and-cut to dynamically add valid inequalities to tighten the relaxation of the problem, and through efficient data structures like sparse matrices to reduce memory and computational costs. Additionally, it distributes the computation over multiple cores, accelerating convergence. Since the Alibaba trace has an 8.5× more job invocation rate than the Google Borg trace, we observe a higher decision-making overhead when invoking jobs following the Alibaba trace. The temporal variations in decision-making overhead are due to the variations in job invocation rate following Google Borg or Alibaba trace.

The delay tolerance component in WaterWise accounts for the communication costs of sending and executing the job to a remote location (including the application package transfer). In Table 3, we present an example result where the home location is Oregon and the results are needed locally after execution at a remote location, it adds only small latency, carbon, and water footprint overhead, expressed

as percentages of the execution time, the carbon footprint during execution, and the water footprint during execution, respectively. Transfer latency takes the majority part in the communication cost.

7 Discussion

Generalization to Different Data Centers. WaterWise's carbon and water optimizations are inherently designed to capture the key sustainability factors that can vary across data centers, such as PUE, carbon emissions, and onsite & offsite water consumption (EWIF, WUE). By incorporating these parameters, WaterWise allows for comprehensive analysis and informed decision-making to reduce environmental impact. Many individual data center parameters such as PUE, CI, EWIF, WUE, and WSF are configurable in the WaterWise framework for the community to gain insights into water and carbon trade-offs. While WaterWise's evaluation focuses on a set of representative parameters, these configurable parameters allow us to evaluate the impact across data centers.

Performance Considerations. WaterWise's primary focus is on co-optimizing carbon and water, instead of pushing the state-of-the-art for scheduling for performance for executing jobs among different regions. WaterWise is complementary to performance-only focused scheduling and does not inherently prohibit integration with such techniques. For any technique that focuses on performance only,WaterWise can be configured to treat performance as a third objective with a configurable weight factor instead of our current delay-tolerance feature, which accounts for allowable scheduling slack based on environmental goals, could serve as a foundation for incorporating performance-driven criteria. This approach can broaden the scope of scheduling systems – the integration of sustainability with traditional computational goals.

Cost Considerations. The primary goal of WaterWise is to demonstrate that carbon and water are competing sustainability metrics. Similar to how performance considerations, incorporating financial costs, such as energy consumption or VM usage, into the optimization objective could dilute the environmental focus and add unnecessary complexity to the model, especially given the variability of cost structures across providers and regions. However, financial costs can be integrated into the optimization goals in future extensions to further enhance the WaterWise.

8 Related Work

Optimizing for performance metrics like latency and throughput, as well as designing energy-efficient hardware and software stacks, have long been central to data center research [3, 7, 21, 22, 43]. However, optimizing solely for energy efficiency is insufficient to minimize the environmental impact of data centers. Sustainability metrics, such as carbon and water footprints, have largely been overlooked in data center research until recently when there has been a growing interest in addressing this gap [25, 31, 32, 42, 55]. Optimizing for energy is not equivalent to optimizing for carbon footprint as the latter also depends on carbon intensity and the embodied carbon of a device. Researchers have proposed strategies to scale resources allocated to applications to match the carbon cost of the energy supply, exploit temporal variation in carbon intensity [26, 50, 56], or incorporate geographical load balancing strategies [34, 37] or both [51]. Focusing solely on the carbon-performance tradeoff can increase application running costs. To address this, GAIA, a carbon-aware scheduler introduced in [27], co-optimizes for performance, carbon emissions, and cost in cloud-based batch processing systems.

Efficient server design using carbon-efficient server components is another opportunity that previous research has explored [2, 17, 20, 40, 54]. Strategical placement of data centers to minimize one or more environmental footprints is another approach that has been proposed [49].

In [32], the authors provide a methodology to estimate the water footprint of AI models and show that spatial and temporal diversity of WUE can be leveraged to reduce the water footprint of AI model training. Water-focused or water-aware research is beginning to gain attention [16, 28, 33, 61]. For example, the water consumption of information retrieval systems is analyzed in [61]. In [33], the authors analyze the carbon and water footprints of AI model inference. However, no prior work systematically exploits the co-optimization of carbon and water footprint – esp. considering the water scarcity factor. As we show in this work, simply saving water consumption is not sufficient – it is equally critical to understand and consider the regional preciousness of water and understand the water generation trade-offs for carbon-friendly energy sources for electricity generation. While the community has access to carbon-aware resource management frameworks, it does not have access to open-source frameworks and open-source water data to develop new carbon-aware and water-aware resource management strategies – this work will bridge that gap.

9 Conclusion

This paper introduced WaterWise, a framework designed to co-optimize the carbon and water footprint of parallel jobs in geographically distributed data centers. This paper comprehensively analyzes and models the carbon and water footprint to reveal new insights and design a novel scheduler. WaterWise provides approx. 21% carbon-footprint savings and approx. 14% water-footprint savings compared to the carbon- and water-unaware scheduler.

Acknowledgment. We thank Yuanchao Xu (our shepherd) and the reviewers for their constructive feedback. This work is supported by NSF Awards 1910601, 2124897.

References

[1] 2024. Water Use and Stress — ourworldindata.org. https://ourworldindata.org/water-use-stress.

[2] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Aditya Sundarrajan, Kiwan Maeng, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon dependencies in datacenter design and management. *ACM SIGENERGY Energy Informatics Review* 3, 3 (2023), 21–26.

[3] Pragati Agrawal and Shrisha Rao. 2014. Energy-aware scheduling of distributed systems. *IEEE Transactions on Automation Science and Engineering* 11, 4 (2014), 1163–1175.

[4] Pradeep Ambati, Noman Bashir, David Irwin, and Prashant Shenoy. 2021. Good things come to those who wait: Optimizing job waiting in the cloud. In *Proceedings of the ACM Symposium on Cloud Computing*. 229–242.

[5] Thomas Anderson, Adam Belay, Mosharaf Chowdhury, Asaf Cidon, and Irene Zhang. 2023. Treehouse: A case for carbon-aware datacenter software. *ACM SIGENERGY Energy Informatics Review* 3, 3 (2023), 64–70.

[6] Clément Auger, Benoit Hilloulin, Benjamin Boisserie, Maël Thomas, Quentin Guignard, and Emmanuel Rozière. 2021. Open-source carbon footprint estimator: Development and university declination. *Sustainability* 13, 8 (2021), 4315.

[7] Mario Bambagini, Mauro Marinoni, Hakan Aydin, and Giorgio Buttazzo. 2016. Energy-aware scheduling for real-time systems: A survey. *ACM Transactions on Embedded Computing Systems (TECS)* 15, 1 (2016), 1–34.

[8] Stephen Boyd and Lieven Vandenberghe. 2004. *Convex optimization*. Cambridge university press.

[9] Thomas Bruckner, L Fulton, E Hertwich, A McKinnon, D Perczyk, J Roy, R Schaeffer, S Schlömer, R Sims, P Smith, and R Wiser. 2014. Technology-specific cost and performance parameters [annex III]. In *Climate change 2014: mitigation of climate change*. Cambridge University Press, 1329–1356.

[10] Shuang Chen, Christina Delimitrou, and José F Martínez. 2019. Parties: Qos-aware resource partitioning for multiple interactive services. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 107–120.

[11] Yue Cheng, Ali Anwar, and Xuejing Duan. 2018. Analyzing alibaba's co-located datacenter workloads. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 292–297.

[12] Benjamin Davy. 2021. Building an AWS EC2 carbon emissions dataset. *Medium, September* (2021).

[13] Benjamin Davy. 2024. Building an AWS EC2 carbon emissions dataset. https://medium.com/teads-engineering/building-an-aws-ec2-carbon-emissions-dataset-3f0fd76c98ac

[14] Khosrow Ebrahimi, Gerard F Jones, and Amy S Fleischer. 2014. A review of data center cooling technology, operating conditions and the corresponding low-grade waste heat recovery opportunities. *Renewable and sustainable energy reviews* 31 (2014), 622–638.

[15] Lieven Eeckhout. 2022. A first-order model to assess computer architecture sustainability. *IEEE Computer Architecture Letters* 21, 2 (2022), 137–140.

[16] Abel U Egbemhenghe, Toluwalase Ojeyemi, Kingsley O Iwuozor, Ebuka Chizitere Emenike, Tolu I Ogunsanya, Stella Ukamaka Anidiobi, and Adewale George Adeniyi. 2023. Revolutionizing water treatment, conservation, and management: Harnessing the power of AI-driven ChatGPT solutions. *Environmental Challenges* 13 (2023), 100782.

[17] Mariam Elgamal, Doug Carmean, Elnaz Ansari, Okay Zed, Ramesh Peri, Srilatha Manne, Udit Gupta, Gu-Yeon Wei, David Brooks, Gage Hills, and Carole-Jean Wu. 2023. Carbon-Efficient Design Optimization for Computing Systems. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems* (Boston, MA, USA) *(HotCarbon '23)*. Association for Computing Machinery, New York, NY, USA, Article 16, 7 pages. https://doi.org/10.1145/3604930.3605712

[18] Michael Ferdman, Almutaz Adileh, Onur Kocberber, Stavros Volos, Mohammad Alisafaee, Djordje Jevdjic, Cansu Kaynak, Adrian Daniel Popescu, Anastasia Ailamaki, and Babak Falsafi. 2012. Clearing the clouds: a study of emerging scale-out workloads on modern hardware. *Acm sigplan notices* 47, 4 (2012), 37–48.

[19] Christodoulos A Floudas and Xiaoxia Lin. 2005. Mixed integer linear programming in process scheduling: Modeling, algorithms, and applications. *Annals of Operations Research* 139 (2005), 131–162.

[20] Eitan Frachtenberg, Ali Heydari, Harry Li, Amir Michael, Jacob Na, Avery Nisbet, and Pierluigi Sarti. 2011. High-efficiency server design. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Seattle, Washington) *(SC '11)*. Association for Computing Machinery, New York, NY, USA, Article 27, 27 pages. https://doi.org/10.1145/2063384.2063420

[21] Jiechao Gao, Haoyu Wang, and Haiying Shen. 2020. Smartly handling renewable energy instability in supporting a cloud datacenter. In *2020 IEEE international parallel and distributed processing symposium (IPDPS)*. IEEE, 769–778.

[22] Íñigo Goiri, Kien Le, Md E Haque, Ryan Beauchea, Thu D Nguyen, Jordi Guitart, Jordi Torres, and Ricardo Bianchini. 2011. Greenslot: scheduling energy consumption in green datacenters. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11.

[23] Google. 2023. *Google 2023 Environmental Repor*. https://www.gstatic.com/gumdrop/sustainability/google-2023-environmental-report.pdf

[24] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 784–799.

[25] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing carbon: The elusive environmental footprint of computing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 854–867.

[26] Walid A Hanafy, Qianlin Liang, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency. *arXiv preprint arXiv:2302.08681* (2023).

[27] Walid A. Hanafy, Qianlin Liang, Noman Bashir, Abel Souza, David Irwin, and Prashant Shenoy. 2024. Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 479–496. https://doi.org/10.1145/3620666.3651374

[28] Mohammad A. Islam, Kishwar Ahmed, Hong Xu, Nguyen H. Tran, Gang Quan, and Shaolei Ren. 2018. Exploiting Spatio-Temporal Diversity for Water Saving in Geo-Distributed Data Centers. *IEEE Transactions on Cloud Computing* 6, 3 (2018), 734–746. https://doi.org/10.1109/TCC.2016.2535201

[29] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K Nurminen, and Zhonghong Ou. 2018. Rapl in action: Experiences in using rapl for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)* 3, 2 (2018), 1–26.

[30] Jacob Leverich and Christos Kozyrakis. 2014. Reconciling high server utilization and sub-millisecond quality-of-service. In *Proceedings of the Ninth European Conference on Computer Systems*. 1–14.

[31] Baolin Li, Rohan Basu Roy, Daniel Wang, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Toward Sustainable HPC: Carbon Footprint Estimation and Environmental Implications of HPC Systems. In *Proceedings of the International Conference for High Performance*

Computing, Networking, Storage and Analysis. 1–15.

[32] Pengfei Li, Jianyi Yang, Mohammad A Islam, and Shaolei Ren. 2023. Making ai less" thirsty": Uncovering and addressing the secret water footprint of ai models. *arXiv preprint arXiv:2304.03271* (2023).

[33] Pengfei Li, Jianyi Yang, Adam Wierman, and Shaolei Ren. 2023. Towards Environmentally Equitable AI via Geographical Load Balancing. arXiv:2307.05494 [cs.AI]

[34] Zhenhua Liu, Minghong Lin, Adam Wierman, Steven Low, and Lachlan L. H. Andrew. 2015. Greening Geographical Load Balancing. *IEEE/ACM Transactions on Networking* 23, 2 (2015), 657–671. https://doi.org/10.1109/TNET.2014.2308295

[35] Jordan Macknick, Robin Newmark, Garvin Heath, and KC Hallett. 2011. Review of operational water consumption and withdrawal factors for electricity generating technologies. (2011).

[36] Jordan Macknick, Robin Newmark, Garvin Heath, and Kathleen C Hallett. 2012. Operational water consumption and withdrawal factors for electricity generating technologies: a review of existing literature. *Environmental Research Letters* 7, 4 (2012), 045802.

[37] Diptyaroop Maji, Ben Pfaff, Vipin P R, Rajagopal Sreenivasan, Victor Firoiu, Sreeram Iyer, Colleen Josephson, Zhelong Pan, and Ramesh K Sitaraman. 2023. Bringing Carbon Awareness to Multi-cloud Application Delivery. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems* (Boston, MA, USA) *(HotCarbon '23)*. Association for Computing Machinery, New York, NY, USA, Article 6, 6 pages. https://doi.org/10.1145/3604930.3605711

[38] Electricity Maps. 2024. *Electricity Maps Live 24/7*. https://app.electricitymaps.com/map

[39] meteologix. 2024. *meteologix Live 24/7*. https://meteologix.com/in

[40] Amirhossein Mirhosseini, Akshitha Sriraman, and Thomas F. Wenisch. 2019. Enhancing Server Efficiency in the Face of Killer Microseconds. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 185–198. https://doi.org/10.1109/HPCA.2019.00037

[41] Stuart Mitchell, Michael OSullivan, and Iain Dunning. 2011. Pulp: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand* 65 (2011).

[42] Steven Gonzalez Monserrate. 2022. The cloud is material: On the environmental impacts of computation and data storage. (2022).

[43] Vinicius Petrucci, Michael A Laurenzano, John Doherty, Yunqi Zhang, Daniel Mosse, Jason Mars, and Lingjia Tang. 2015. Octopus-man: Qos-driven task management for heterogeneous multicores in warehouse-scale computers. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 246–258.

[44] Paul Reig. 2013. What's the Difference Between Water Use and Water Consumption? https://www.wri.org/insights/whats-difference-between-water-use-and-water-consumption.

[45] Paul Reig, Tianyi Luo, Eric Christensen, and Julie Sinistore. 2020. Guidance for calculating water use embedded in purchased electricity. *World Resources Institute* (2020).

[46] Hannah Ritchie, Pablo Rosado, and Max Roser. 2024. Electricity mix. *Our World in Data* (2024).

[47] Arman Shehabi, Sarah Smith, Dale Sartor, Richard Brown, Magnus Herrlin, Jonathan Koomey, Eric Masanet, Nathaniel Horner, Inês Azevedo, and William Lintner. 2016. United states data center energy usage report. (2016).

[48] Md Abu Bakar Siddik, Arman Shehabi, and Landon Marston. 2021. The environmental footprint of data centers in the United States. *Environmental Research Letters* 16, 6 (2021), 064017.

[49] Md Abu Bakar Siddik, Arman Shehabi, and Landon Marston. 2021. The environmental footprint of data centers in the United States. *Environmental Research Letters* 16, 6 (may 2021), 064017. https://doi.org/10.1088/1748-9326/abfba1

[50] Abel Souza, Noman Bashir, Jorge Murillo, Walid Hanafy, Qianlin Liang, David Irwin, and Prashant Shenoy. 2023. Ecovisor: A Virtual Energy System for Carbon-Efficient Applications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 252–265. https://doi.org/10.1145/3575693.3575709

[51] Thanathorn Sukprasert, Abel Souza, Noman Bashir, David Irwin, and Prashant Shenoy. 2024. On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 924–941. https://doi.org/10.1145/3627703.3650079

[52] Huangshi Tian, Yunchuan Zheng, and Wei Wang. 2019. Characterizing and synthesizing task dependencies of data-parallel jobs in alibaba cloud. In *Proceedings of the ACM Symposium on Cloud Computing*. 139–151.

[53] J. Treibig, G. Hager, and G. Wellein. 2010. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proceedings of PSTI2010, the First International Workshop on Parallel Software Tools and Tool Infrastructures*. San Diego CA.

[54] Jaylen Wang, Daniel S. Berger, Fiodar Kazhamiaka, Celine Irvene, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warrier, Chetan Bansal, Jonathan Stern, Ricardo Bianchini, and Akshitha Sriraman. 2024. Designing Cloud Servers for Lower Carbon. In *ISCA*. https://www.microsoft.com/en-us/research/publication/designing-cloud-servers-for-lower-carbon/

[55] Jaylen Wang, Udit Gupta, and Akshitha Sriraman. 2023. Peeling Back the Carbon Curtain: Carbon Optimization Challenges in Cloud Computing. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems* (Boston, MA, USA) *(HotCarbon '23)*. Association for Computing Machinery, New York, NY, USA, Article 8, 7 pages. https://doi.org/10.1145/3604930.3605718

[56] Philipp Wiesner, Ilja Behnke, Dominik Scheinert, Kordian Gontarska, and Lauritz Thamsen. 2021. Let's wait awhile: how temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference* (Québec city, Canada) *(Middleware '21)*. Association for Computing Machinery, New York, NY, USA, 260–272. https://doi.org/10.1145/3464298.3493399

[57] John Wilkes. 2020. Yet more Google compute cluster trace data. Google research blog. Posted at https://ai.googleblog.com/2020/04/yet-more-google-compute-cluster-trace.html..

[58] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin S. Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. 2022. Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems* 4 (2022), 795–813.

[59] Xusheng Zhan, Yungang Bao, Christian Bienia, and Kai Li. 2017. PARSEC3. 0: A multicore benchmark suite with network stacks and SPLASH-2X. *ACM SIGARCH Computer Architecture News* 44, 5 (2017), 1–16.

[60] Di Zhang, Dong Dai, and Bing Xie. 2022. Schedinspector: A batch job scheduling inspector using reinforcement learning. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*. 97–109.

[61] Guido Zuccon, Harrisen Scells, and Shengyao Zhuang. 2023. Beyond CO2 emissions: The overlooked impact of water consumption of information retrieval models. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*. 283–289.

A Artifact Appendix

A.1 Abstract

WaterWise is an innovative job scheduling framework designed to co-optimize the carbon and water footprints of geographically distributed data centers. It leverages a Mixed Integer Linear Programming (MILP) approach to dynamically adjust scheduling based on temporal and spatial variations in carbon intensity and water intensity. WaterWise balances the optimization goals with low decision-making overhead and delay latency by incorporating delay tolerance and slack management. This open-source artifact is available at:

https://zenodo.org/records/14219862

The artifact includes the following:

- Data for sustainable metrics, including WUE, EWIF, WSF, and carbon intensity.
- The profiling data for energy consumption, and execution time of parallel applications.
- Scripts and data for generating motivation plots.
- Scripts to setting up and evaluating the WaterWise.
- Scripts for implementing and comparing different optimizers within WaterWise.

A.2 Artifact check-list (meta-information)

- **Algorithm:** A MILP-based algorithm for scheduling parallel workloads.
- **Program:** Optimizers designed to reduce carbon and water footprints.
- **Data set:** Carbon intensity and EWIF are sourced from Electricity Map, while WUE is calculated based on wet bulb temperature data from Meteologix. More details are provided in the artifact.
- **Run-time environment:** Python 3.8 with required Python packages.
- **Hardware:** No specific hardware is required for simulation. Profiling data is collected from AWS EC2 m5.metal bare-metal machines.
- **Execution:** Follow the instructions in README.md.
- **Metrics:** Includes execution time, energy consumption, carbon footprint, water footprint for a single job, and transfer latency between regions.
- **Output:** Logs are generated, including carbon and water footprint, execution time, and overhead. Refer to README.md for more details.
- **Experiments:** (1). Experiments using Google Borg trace. (2). Experiments using Alibaba VM trace. (3). Sensitivity and robustness analysis.
- **How much disk space required (approximately)?:** More than 10 GB.
- **How much time is needed to prepare workflow (approximately)?:** 5 mins.
- **How much time is needed to complete experiments (approximately)?:** More than 1 hour.
- **Publicly available?:** Yes
- **Archived (DOI)?:** https://zenodo.org/records/14219862

A.3 Description

This artifact provides the framework of WaterWise, designed to execute parallel jobs on cloud servers. WaterWise simulates job execution using industry-standard VM cloud traces. The inputs include carbon intensity, water intensity, and job data. After configuring the parameters of WaterWise, the framework performs a 10-day simulation based on the VM trace (230,000 jobs from the Google Borg trace). WaterWise's Decision Controller leverages a MILP algorithm to determine which jobs are sent to which regions, generating detailed log files for reference.

A.4 Installation

To set up the WaterWise artifact, refer to the README.md file included in the artifact. The README.md provides detailed installation instructions. Additionly, several Python libraries need to be installed to run the artifact.

A.5 Expected results

Main figures are expected to be reproduced from the data in the artifact.